

Ejercicio 13 — hashCode, HashMap y contrato equals/hashCode

Parte 1 — Implementación de hashCode en Java

Object.hashCode()

Es un método nativo declarado como:

```
public native int hashCode();
```

Por defecto devuelve un valor entero derivado de la identidad del objeto, no de su contenido. Históricamente se asoció con la dirección de memoria, pero las JVM modernas (HotSpot) usan estrategias configurables mediante el flag -XX:hashCode:

- Política 0: número pseudoaleatorio global (Park-Miller RNG).
- Política 5 (default actual en HotSpot): generador xorshift por hilo.
- Otras políticas: secuencial, basado en dirección, constante (0), etc.

El valor calculado se cachea en el header del objeto (mark word) la primera vez que se invoca, garantizando que en invocaciones sucesivas devuelva siempre lo mismo para esa instancia.

Consecuencia clave: dos objetos new Object() distintos siempre producen hashes distintos (salvo colisión), aunque "representen lo mismo".

Integer.hashCode()

```
@Override
public int hashCode() {
    return Integer.hashCode(value);
}

public static int hashCode(int value) {
    return value;
}
```

Es trivial: el hash es el propio valor entero. Tiene sentido porque un int ya cabe en 32 bits, exactamente el tamaño que devuelve hashCode(), y garantiza que new Integer(5).equals(new Integer(5)) implique mismo hash.

String.hashCode()

```
public int hashCode() {
    int h = hash;
```

```

if (h == 0 && value.length > 0) {
    char val[] = value;
    for (int i = 0; i < value.length; i++) {
        h = 31 * h + val[i];
    }
    hash = h;
}
return h;
}

```

Fórmula: $s[0] \cdot 31^{(n-1)} + s[1] \cdot 31^{(n-2)} + \dots + s[n-1]$.

¿Por qué son diferentes las implementaciones?

Porque cada clase tiene una noción distinta de igualdad y hashCode debe ser consistente con equals:

- Object: igualdad por identidad (==). Estrategia de hash: identidad del objeto en memoria.
- Integer: igualdad por mismo valor numérico. Estrategia de hash: el valor mismo.
- String: igualdad por misma secuencia de caracteres. Estrategia de hash: polinomio sobre los caracteres.

Si Integer heredara el hashCode de Object, dos Integer(5) distintos darían hashes distintos, rompiendo la lógica de los HashMap. Por eso tanto Integer como String sobrescriben hashCode (y equals) para reflejar igualdad por contenido.

Parte 2 — Estructura interna de un HashMap

Un HashMap en Java mantiene internamente un array de buckets (Node<K,V>[] table) con tamaño inicial 16 (potencia de 2) y factor de carga por defecto 0.75. Cada bucket puede contener:

1. Vacío (null).
2. Un solo Node (par clave-valor).
3. Una lista enlazada de Nodes (en caso de colisión).
4. Un árbol rojo-negro (TreeNode) cuando una lista crece a 8 o más elementos y la tabla tiene al menos 64 buckets — esto se llama treeification y se introdujo en Java 8.

Estructura de un Node:

```

static class Node<K,V> {
    final int hash;

```

```

    final K key;
    V value;
    Node<K,V> next;
}

```

Cálculo del índice

```

hash = key.hashCode() ^ (key.hashCode() >>> 16);
index = (n - 1) & hash;

```

El XOR con los bits altos (spread function) mejora la distribución cuando n es chico. La operación $(n-1) \& \text{hash}$ equivale a $\text{hash} \% n$ pero más rápida, y requiere que n sea potencia de 2.

Inserción de "Hola", "HolaMundo", "HashMap", "Colecciones"

Valores reales de hashCode() devueltos por Java:

```

- "Hola":      hashCode = 2255068    spread = 2255102    índice = 14
- "HolaMundo": hashCode = -1191400619 spread = -1191439447 índice = 9
- "HashMap":   hashCode = -1932803762 spread = -1932833403 índice = 5
- "Colecciones": hashCode = -1872965711 spread = -1872994323 índice = 13

```

Diagrama del estado final

```

table[0] -> null
table[1] -> null
table[2] -> null
table[3] -> null
table[4] -> null
table[5] -> Node("HashMap")
table[6] -> null
table[7] -> null
table[8] -> null
table[9] -> Node("HolaMundo")
table[10] -> null
table[11] -> null
table[12] -> null
table[13] -> Node("Colecciones")
table[14] -> Node("Hola")
table[15] -> null

```

Parte 3 — equals y hashCode para Alumno

Implementación

```
import java.util.Objects;

public class Alumno {
    private int id;
    private String fullName;
    private String email;

    public Alumno(int id, String fullName, String email) {
        this.id = id;
        this.fullName = fullName;
        this.email = email;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Alumno)) return false;
        Alumno other = (Alumno) o;
        return this.id == other.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

Decisión de diseño

La identidad lógica de un Alumno se define por su id, que actúa como clave primaria del dominio. fullName y email pueden cambiar (corrección de tipeo en el nombre, cambio de mail laboral, etc.), por lo que no son aptos para definir igualdad: si los incluyera en equals/hashCode, un Alumno modificado quedaría "perdido" en su bucket del HashMap y nunca más se lo podría encontrar con get().

Contrato general de hashCode que se respeta

1. Consistencia interna durante una ejecución: invocar hashCode() varias veces sobre el mismo

objeto debe devolver siempre el mismo entero, siempre que no haya cambiado la información usada en equals. Aquí se cumple porque id no debería mutarse.

2. Coherencia con equals (el punto crítico): si `a.equals(b)` es true, entonces `a.hashCode() == b.hashCode()` obligatoriamente. Aquí se cumple porque ambos métodos dependen exclusivamente de id. Violar esto rompería HashMap: dos alumnos "iguales" podrían terminar en buckets distintos, y `containsKey` devolvería false para una clave que en realidad sí está.

3. No es necesaria la recíproca: dos objetos con el mismo hashCode no tienen que ser equals. Las colisiones son inevitables (el espacio de hashes es 2^{32} pero el de objetos es infinito) y HashMap ya las maneja con listas o árboles en cada bucket. Lo que sí es deseable, aunque no obligatorio, es que objetos desiguales tiendan a producir hashes distintos para mantener los buckets balanceados.

4. Inmutabilidad de la clave: los campos usados por hashCode y equals no deben modificarse mientras el objeto esté en una estructura basada en hashing. Como id representa identidad, no debería mutarse nunca.

Ejercicio 14 — Comparación de estrategias de resolución de colisiones

Conjunto de claves (en orden de inserción): 45, 12, 37, 82, 29, 54, 31, 76, 18, 93, 11, 68 (N = 12).

1) Tamaño de la tabla

Para que la comparación entre las tres estrategias sea justa, se elige un único tamaño de tabla: $M = 17$.

Justificación:

- Es primo, lo que mejora la distribución del hash con la función de división y minimiza agrupamientos.
- Para sondeo cuadrático, cuando M es primo se garantiza que el sondeo $(h + i^2) \bmod M$ recorra al menos $\lceil M/2 \rceil = 9$ posiciones distintas antes de repetir, suficiente para nuestros 12 elementos sin riesgo de loops infinitos prematuros mientras la tabla esté menos del 50% llena.
- Da un factor de carga $\alpha = 12/17 \approx 0.706$, adecuado para direccionamiento abierto (idealmente $\alpha \leq 0.7$) y aceptable para encadenamiento separado (que tolera $\alpha \approx 1$).
- Usar el mismo M en las tres estrategias permite comparar resultados sin sesgo introducido por el tamaño.

2) Función hash

Para las tres estrategias: $h(k) = k \bmod 17$.

Cálculo para cada clave:

- $45 \bmod 17 = 11$
- $12 \bmod 17 = 12$
- $37 \bmod 17 = 3$
- $82 \bmod 17 = 14$
- $29 \bmod 17 = 12$
- $54 \bmod 17 = 3$
- $31 \bmod 17 = 14$
- $76 \bmod 17 = 8$
- $18 \bmod 17 = 1$
- $93 \bmod 17 = 8$
- $11 \bmod 17 = 11$
- $68 \bmod 17 = 0$

Pares que colisionan en su hash inicial: (12, 29), (37, 54), (82, 31), (76, 93), (45, 11).

3) Estado final por estrategia

A) Sondeo lineal — $h(k, i) = (h(k) + i) \bmod 17$

Traza de inserciones:

- 45 ($h=11$): pos 11 libre. Posición final 11. Colisiones: 0.
- 12 ($h=12$): pos 12 libre. Posición final 12. Colisiones: 0.
- 37 ($h=3$): pos 3 libre. Posición final 3. Colisiones: 0.
- 82 ($h=14$): pos 14 libre. Posición final 14. Colisiones: 0.
- 29 ($h=12$): 12 ocupada, 13 libre. Posición final 13. Colisiones: 1.
- 54 ($h=3$): 3 ocupada, 4 libre. Posición final 4. Colisiones: 1.
- 31 ($h=14$): 14 ocupada, 15 libre. Posición final 15. Colisiones: 1.
- 76 ($h=8$): pos 8 libre. Posición final 8. Colisiones: 0.
- 18 ($h=1$): pos 1 libre. Posición final 1. Colisiones: 0.
- 93 ($h=8$): 8 ocupada, 9 libre. Posición final 9. Colisiones: 1.
- 11 ($h=11$): 11 ocupada, 12 ocupada, 13 ocupada, 14 ocupada, 15 ocupada, 16 libre. Posición final 16. Colisiones: 5.
- 68 ($h=0$): pos 0 libre. Posición final 0. Colisiones: 0.

Estado final:

pos:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key:	68	18	-	37	54	-	-	-	76	93	-	45	12	29	82	31	11

Total de colisiones: 9.

B) Sondeo cuadrático — $h(k, i) = (h(k) + i^2) \bmod 17$

Traza de inserciones:

- 45 (h=11): pos 11 libre. Final 11. Colisiones: 0.
- 12 (h=12): pos 12 libre. Final 12. Colisiones: 0.
- 37 (h=3): pos 3 libre. Final 3. Colisiones: 0.
- 82 (h=14): pos 14 libre. Final 14. Colisiones: 0.
- 29 (h=12): 12 ocupada, $12+1=13$ libre. Final 13. Colisiones: 1.
- 54 (h=3): 3 ocupada, $3+1=4$ libre. Final 4. Colisiones: 1.
- 31 (h=14): 14 ocupada, $14+1=15$ libre. Final 15. Colisiones: 1.
- 76 (h=8): pos 8 libre. Final 8. Colisiones: 0.
- 18 (h=1): pos 1 libre. Final 1. Colisiones: 0.
- 93 (h=8): 8 ocupada, $8+1=9$ libre. Final 9. Colisiones: 1.
- 11 (h=11): 11 ocupada, $11+1=12$ ocupada, $11+4=15$ ocupada, $11+9=20 \bmod 17=3$ ocupada, $11+16=27 \bmod 17=10$ libre. Final 10. Colisiones: 4.
- 68 (h=0): pos 0 libre. Final 0. Colisiones: 0.

Estado final:

pos:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
key:	68	18	-	37	54	-	-	-	76	93	11	45	12	29	82	31	-

Total de colisiones: 8.

C) Encadenamiento separado

Cada clave va a su bucket sin sondeo; las colisiones se acumulan al frente de la lista (estilo HashMap de Java).

Inserciones que producen colisión (bucket ya ocupado): 29, 54, 31, 93, 11. Total de colisiones: 5.

Estado final (cada lista mostrada del frente al fondo, es decir del más reciente al más viejo):

[0]	->	68
[1]	->	18
[2]	->	null
[3]	->	54 -> 37
[4]	->	null
[5]	->	null

[6] -> null
[7] -> null
[8] -> 93 -> 76
[9] -> null
[10] -> null
[11] -> 11 -> 45
[12] -> 29 -> 12
[13] -> null
[14] -> 31 -> 82
[15] -> null
[16] -> null

4) Resumen de colisiones

- Sondeo lineal: 9.
- Sondeo cuadrático: 8.
- Encadenamiento separado: 5.

5) Promedio de comparaciones — búsqueda exitosa

Para cada clave, se cuentan las celdas examinadas hasta encontrarla, partiendo de $h(k)$.
Coincide con los pasos dados en la inserción.

Sondeo lineal

Comparaciones por clave: 45=1, 12=1, 37=1, 82=1, 29=2, 54=2, 31=2, 76=1, 18=1, 93=2, 11=6, 68=1.

Suma = 21. Promedio = $21/12 \approx 1.75$.

Sondeo cuadrático

Comparaciones por clave: 45=1, 12=1, 37=1, 82=1, 29=2, 54=2, 31=2, 76=1, 18=1, 93=2, 11=5, 68=1.

Suma = 20. Promedio = $20/12 \approx 1.67$.

Encadenamiento separado

Como las claves nuevas se insertan al frente, la última en llegar tiene costo 1 y la primera (al fondo) tiene costo igual al largo del bucket.

- Bucket 0: 68 -> 1 comp.

- Bucket 1: 18 -> 1 comp.
- Bucket 3: 54=1, 37=2 -> total 3.
- Bucket 8: 93=1, 76=2 -> total 3.
- Bucket 11: 11=1, 45=2 -> total 3.
- Bucket 12: 29=1, 12=2 -> total 3.
- Bucket 14: 31=1, 82=2 -> total 3.

Suma = 1+1+3+3+3+3+3 = 17. Promedio = $17/12 \approx 1.42$.

6) Promedio de comparaciones — búsqueda no exitosa

Se utiliza una clave testigo por cada uno de los 17 buckets posibles ($h = 0$ a 16). Esto da el promedio teórico real en lugar de muestrear unos pocos casos. Cuando una clave candidata coincidía con alguna ya insertada (por ejemplo $k=18$), se la sustituye por otra con el mismo h pero que no esté en la tabla.

Claves testigo elegidas:

- $h=0$: 17. $h=1$: 35. $h=2$: 19. $h=3$: 20.
- $h=4$: 21. $h=5$: 22. $h=6$: 23. $h=7$: 24.
- $h=8$: 25. $h=9$: 26. $h=10$: 27. $h=11$: 28.
- $h=12$: 46. $h=13$: 30. $h=14$: 48. $h=15$: 32.
- $h=16$: 33.

Sondeo lineal

Una búsqueda fallida recorre celdas hasta encontrar la primera vacía. Desde cada h :

- $h=0$: 0(o)→1(o)→2(v). 3 comp.
- $h=1$: 1(o)→2(v). 2 comp.
- $h=2$: 2(v). 1 comp.
- $h=3$: 3(o)→4(o)→5(v). 3 comp.
- $h=4$: 4(o)→5(v). 2 comp.
- $h=5$: 5(v). 1 comp.
- $h=6$: 6(v). 1 comp.
- $h=7$: 7(v). 1 comp.
- $h=8$: 8(o)→9(o)→10(v). 3 comp.
- $h=9$: 9(o)→10(v). 2 comp.
- $h=10$: 10(v). 1 comp.
- $h=11$: 11(o)→12(o)→13(o)→14(o)→15(o)→16(o)→0(o)→1(o)→2(v). 9 comp.
- $h=12$: 12(o)→13(o)→14(o)→15(o)→16(o)→0(o)→1(o)→2(v). 8 comp.
- $h=13$: 13(o)→14(o)→15(o)→16(o)→0(o)→1(o)→2(v). 7 comp.
- $h=14$: 14(o)→15(o)→16(o)→0(o)→1(o)→2(v). 6 comp.

- h=15: 15(o)→16(o)→0(o)→1(o)→2(v). 5 comp.
- h=16: 16(o)→0(o)→1(o)→2(v). 4 comp.

Suma = 3+2+1+3+2+1+1+1+3+2+1+9+8+7+6+5+4 = 59. Promedio = 59/17 ≈ 3.47.

Sondeo cuadrático

El sondeo es h, h+1, h+4, h+9, h+16 (mod 17). Se recorre hasta la primera celda vacía:

- h=0: 0(o)→1(o)→4(o)→9(o)→16(v). 5 comp.
- h=1: 1(o)→2(v). 2 comp.
- h=2: 2(v). 1 comp.
- h=3: 3(o)→4(o)→7(v). 3 comp.
- h=4: 4(o)→5(v). 2 comp.
- h=5: 5(v). 1 comp.
- h=6: 6(v). 1 comp.
- h=7: 7(v). 1 comp.
- h=8: 8(o)→9(o)→12(o)→0(o)→7(v). 5 comp.
- h=9: 9(o)→10(o)→13(o)→1(o)→8(o)→0(o)→11(o)→7(v). 8 comp.
- h=10: 10(o)→11(o)→14(o)→2(v). 4 comp.
- h=11: 11(o)→12(o)→15(o)→3(o)→10(o)→2(v). 6 comp.
- h=12: 12(o)→13(o)→16(v). 3 comp.
- h=13: 13(o)→14(o)→0(o)→5(v). 4 comp.
- h=14: 14(o)→15(o)→1(o)→6(v). 4 comp.
- h=15: 15(o)→16(v). 2 comp.
- h=16: 16(v). 1 comp.

Suma = 5+2+1+3+2+1+1+1+5+8+4+6+3+4+4+2+1 = 53. Promedio = 53/17 ≈ 3.12.

Encadenamiento separado

Una búsqueda fallida recorre toda la lista del bucket. Si el bucket está vacío, son 0 comparaciones.

Tamaño de cada lista: bucket 0=1, 1=1, 2=0, 3=2, 4=0, 5=0, 6=0, 7=0, 8=2, 9=0, 10=0, 11=2, 12=2, 13=0, 14=2, 15=0, 16=0.

Suma = 1+1+0+2+0+0+0+0+2+0+0+2+2+0+2+0+0 = 12. Promedio = 12/17 ≈ 0.71.

Equivalentemente, este número coincide con $\alpha = n/m = 12/17 \approx 0.706$, que es el resultado teórico clásico para encadenamiento.

7) Comparación y conclusión

Resumen final:

- Sondeo lineal: 9 colisiones, exitosa 1.75, fallida 3.47.
- Sondeo cuadrático: 8 colisiones, exitosa 1.67, fallida 3.12.
- Encadenamiento separado: 5 colisiones, exitosa 1.42, fallida 0.71.

Conclusión: encadenamiento separado ganó en las tres métricas con este conjunto de datos. Las razones son:

1. Las colisiones no se "contagian". En sondeo lineal, el clustering primario hace que cualquier choque empeore inserciones posteriores aunque sus hashes originales no estén relacionados. El ejemplo dramático es la clave 11: su hash 11 ya estaba ocupado por 45, pero además se chocó con el cluster formado por las posiciones 12, 13, 14 y 15, y terminó pagando 5 colisiones para una clave que en principio solo competía con 45. El sondeo cuadrático mitiga parte del problema pero no lo elimina: en la inserción de 11 aún paga 4 colisiones.
2. La búsqueda fallida es donde la diferencia es más fuerte. En direccionamiento abierto, una búsqueda por una clave inexistente cuyo hash cae en una zona densa debe sondear hasta encontrar un hueco, y ese hueco puede estar muy lejos por el clustering. En lineal, una búsqueda fallida que arranca en la posición 11 examina 9 celdas. En encadenamiento, una búsqueda fallida nunca examina más que el largo del bucket, que en este caso es 2 como máximo.
3. El factor de carga es el verdadero protagonista. A $\alpha \approx 0.7$, encadenamiento ya es claramente mejor; si tuviéramos más claves (digamos 15 o 16), el direccionamiento abierto se degradaría aún más rápido mientras que encadenamiento se mantendría con buckets de 2 a 3 elementos en promedio.

Salvedad importante: estos resultados se miden en cantidad de comparaciones, que es la métrica académica estándar. En la práctica (por ejemplo en JVMs modernas), el sondeo lineal puede ser competitivo o incluso superior gracias a la localidad de caché: recorrer celdas contiguas en un array es mucho más barato que saltar por punteros entre nodos. Por eso HashMap de Java combina ambos enfoques: encadenamiento como base, pero con árboles rojo-negros cuando los buckets crecen demasiado, y rehashing agresivo cuando α supera 0.75.